UDF
CURSO DE CIÊNCIA DA COMPUTAÇÃO

MAURÍCIO CUNHA
ADRYAN DA SILVA
DANIEL AYAN
JOÃO PEDRO FONSECA
GABRIEL DOS SANTOS

PROJETO DE BANCO DE DADOS:
SISTEMA DE GERENCIAMENTO DA ACADEMIA BOAFORMA

Brasília-DF
2026

MAURÍCIO CUNHA

ADRYAN DA SILVA

DANIEL AYAN

JOÃO PEDRO FONSECA

GABRIEL DOS SANTOS

PROJETO DE BANCO DE DADOS:
SISTEMA DE GERENCIAMENTO DA ACADEMIA BOAFORMA

Projeto final apresentado à disciplina de Modelagem de Banco de Dados do curso de Ciência da Computação da UDF Centro Universitário, como requisito parcial para a composição da nota do semestre 2026.2.

Professora: Profa. MSc. Josyane Lannes Florenzano de Souza

Brasília-DF

2026

SUMÁRIO

1 INTRODUÇÃO 3
2 MODELAGEM CONCEITUAL 4
2.1 Descrição do cenário (minimundo) 4
2.2 Modelo Entidade-Relacionamento (MER) 5
3 MODELAGEM LÓGICA 8
3.1 Diagrama Entidade-Relacionamento (DER) 8
3.2 Etapas da iteração dentro da modelagem 9
4 MODELAGEM FÍSICA 12
4.1 Script SQL com explicação do código 12
5 CONSULTAS SQL 16
5.1 Consultas básicas 16
5.2 Consultas intermediárias 18
5.3 Consultas avançadas: triggers e stored procedures 27
6 CONSIDERAÇÕES FINAIS 33
REFERÊNCIAS 34
APÊNDICE A – SCRIPT DE CRIAÇÃO DO BANCO 35
APÊNDICE B – COMO RODAR O PROJETO NA APRESENTAÇÃO 38

1 INTRODUÇÃO

Este trabalho apresenta o projeto de um banco de dados relacional para a **Academia BoaForma**, uma academia de ginástica fictícia que nós criamos para a disciplina de Modelagem de Banco de Dados. Escolhemos esse tema porque muitas academias de bairro ainda controlam os alunos, as mensalidades e as fichas de treino em caderno ou em planilha, o que acaba gerando informação repetida, erro na cobrança e retrabalho.

O projeto seguiu as etapas vistas em aula. Primeiro fizemos a **modelagem conceitual**, com a descrição do cenário (minimundo) e o Modelo Entidade-Relacionamento (MER) desenhado no brModelo. Depois veio a **modelagem lógica**, com o Diagrama Entidade-Relacionamento (DER) feito no MySQL Workbench e a explicação das decisões que tomamos em cada iteração. Em seguida fizemos a **modelagem física**, que é o script SQL que cria o banco de verdade, e por último as **consultas SQL**, começando pelas mais simples e terminando com trigger e stored procedure.

As ferramentas usadas foram o brModelo 3.31 (para o MER), o MySQL Workbench 8.0 (para o DER e para rodar os scripts) e o MySQL Server como Sistema Gerenciador de Banco de Dados (SGBD). Os arquivos dos diagramas e os scripts `.sql` foram entregues junto com este documento, e o passo a passo para rodar tudo está no Apêndice B.

2 MODELAGEM CONCEITUAL

A modelagem conceitual é a primeira etapa do projeto. Nela a gente descreve o problema do mundo real sem pensar ainda em tabela ou em comando SQL, só em quais informações o sistema precisa guardar e como elas se relacionam (HEUSER, 2009).

2.1 Descrição do cenário (minimundo)

A Academia BoaForma quer um sistema para organizar as suas tarefas do dia a dia. O sistema deve cadastrar os alunos, os instrutores, os planos vendidos, as modalidades e as turmas, além de controlar os pagamentos das mensalidades, a frequência dos alunos e as fichas de treino.

Cada aluno é identificado por um código, e o sistema deve guardar o seu nome, CPF, data de nascimento, sexo, telefone, e-mail, endereço (rua, número, bairro e cidade), a data da matrícula e a situação, que pode ser ativo ou inativo. Todo aluno assina um único plano, e um mesmo plano pode ser assinado por vários alunos.

Os planos têm um código, nome, descrição, valor mensal e duração em meses. Hoje a academia trabalha com os planos mensal, trimestral, semestral e anual, e quanto maior o plano, menor o valor da mensalidade.

Os instrutores são os profissionais de Educação Física da academia. Para cada um o sistema deve guardar código, nome, CPF, CREF (registro no Conselho Regional de Educação Física), telefone, e-mail, salário e data de admissão.

A academia oferece modalidades como musculação, spinning, pilates, yoga, funcional e zumba. Cada modalidade pode ter várias turmas, e cada turma acontece em um dia da semana e horário, tem um número máximo de vagas e é dada por um único instrutor. Um aluno pode se inscrever em várias turmas e uma turma pode ter vários alunos, e o sistema precisa guardar a data em que o aluno se inscreveu. Quando a turma estiver lotada, o sistema não pode aceitar mais inscrições.

Todo mês o aluno paga a mensalidade. Cada pagamento registra a data em que foi pago, o mês de referência, o valor e a forma de pagamento (PIX, dinheiro, cartão de crédito ou cartão de débito). Um aluno pode ter vários pagamentos, mas

cada pagamento é de um aluno só. Se um aluno que estava inativo fizer um pagamento, ele deve voltar a ficar ativo.

Na entrada da academia tem uma catraca, e cada vez que o aluno passa por ela é gerado um registro de frequência com a data e a hora de entrada.

Por fim, os instrutores montam fichas de treino para os alunos. Cada ficha tem data de criação e um objetivo (hipertrofia, emagrecimento, etc.), pertence a um aluno e é montada por um instrutor. Uma ficha tem um ou mais exercícios, e o mesmo exercício pode aparecer em várias fichas. Para cada exercício da ficha é preciso guardar o número de séries, de repetições e a carga em quilos. Cada exercício tem um nome e o grupo muscular que ele trabalha.

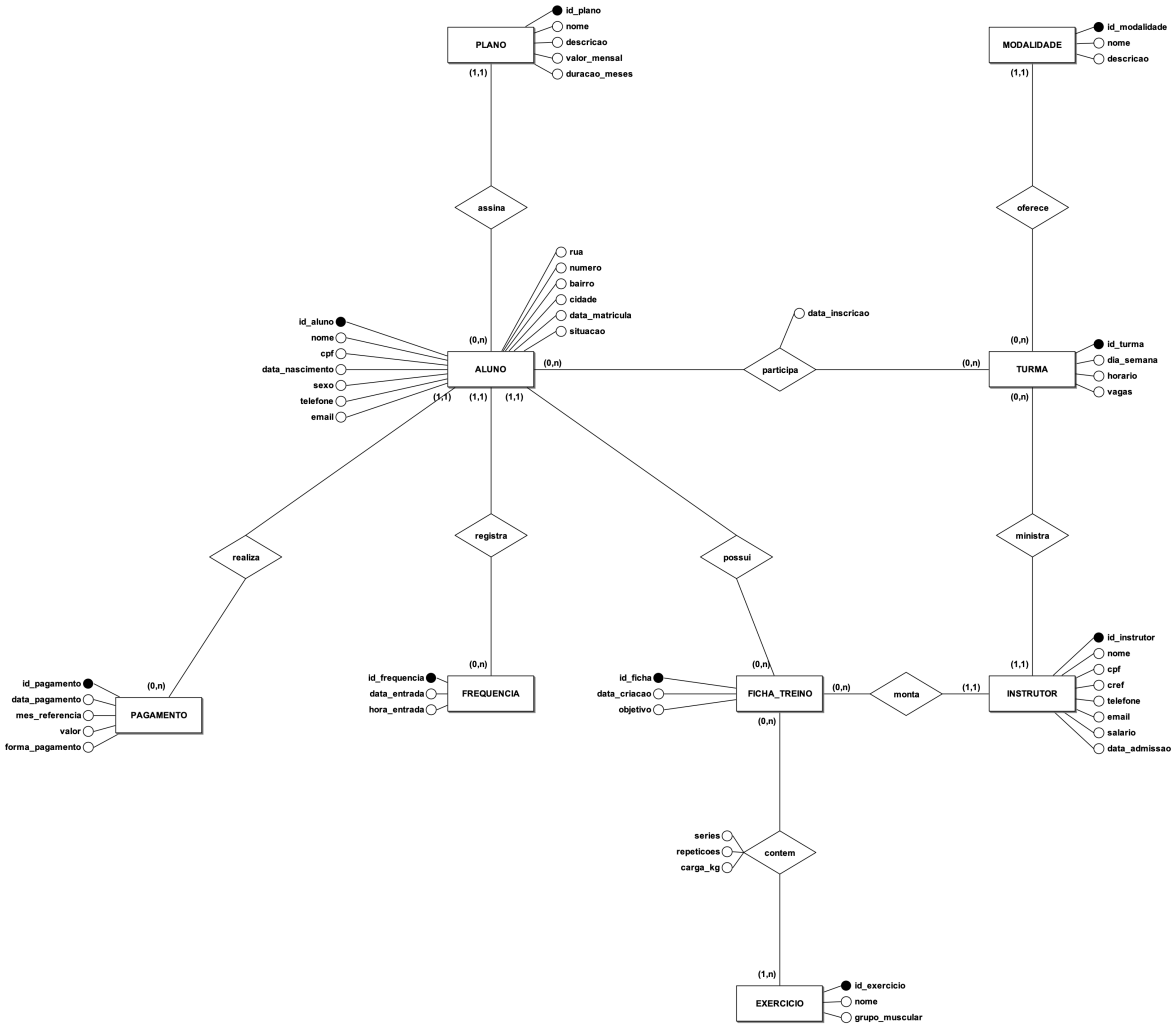
A partir do texto, separamos as principais **regras de negócio** que o banco precisa respeitar:

- um aluno assina exatamente um plano;
- um aluno pode participar de nenhuma ou de várias turmas, e uma turma pode ter nenhum ou vários alunos, respeitando o limite de vagas;
- cada turma pertence a uma modalidade e é dada por um instrutor;
- um aluno pode ter nenhum ou vários pagamentos, registros de frequência e fichas de treino;
- toda ficha de treino precisa ter pelo menos um exercício;
- o CPF do aluno, o CPF do instrutor e o CREF não podem se repetir;
- o aluno não é apagado quando para de treinar, ele só fica com a situação INATIVO, para não perder o histórico.

2.2 Modelo Entidade-Relacionamento (MER)

Com a descrição pronta, fomos marcando no texto os substantivos importantes (que viraram **entidades**), as características de cada um (os **atributos**) e os verbos que ligam uma coisa na outra (os **relacionamentos**). O MER foi desenhado no brModelo, que usa a notação de Peter Chen: o retângulo é a entidade, o losango é o relacionamento e as bolinhas são os atributos. A bolinha pintada de preto é o identificador da entidade, que depois vira a chave primária.

Figura 1 – Modelo Entidade-Relacionamento (MER) da Academia BoaForma



Fonte: Elaborado pelos autores (2026).

O modelo ficou com nove entidades: ALUNO, PLANO, INSTRUTOR, MODALIDADE, TURMA, PAGAMENTO, FREQUENCIA, FICHA_TREINO e EXERCICIO. As cardinalidades aparecem no formato (mínimo, máximo) do lado de cada entidade. Por exemplo, o (1,1) do lado de PLANO no relacionamento "assina" quer dizer que um aluno assina no mínimo um e no máximo um plano, e o (0,n) do lado de ALUNO quer dizer que um plano pode ter de zero a vários alunos. O Quadro 1 resume todos os relacionamentos.

Quadro 1 – Relacionamentos e cardinalidades do MER

Relacionamento	Entidades	Tipo	Como ler
assina	ALUNO e PLANO	N:1	Um aluno assina (1,1) plano; um plano tem (0,n) alunos.
participa	ALUNO e TURMA	N:N	Um aluno participa de (0,n) turmas; uma turma tem (0,n) alunos. Tem o atributo data_inscricao.
oferece	MODALIDADE e TURMA	1:N	Uma turma é de (1,1) modalidade; uma modalidade tem (0,n) turmas.

Relacionamento	Entidades	Tipo	Como ler
ministra	INSTRUTOR e TURMA	1:N	Uma turma é dada por (1,1) instrutor; um instrutor dá (0,n) turmas.
realiza	ALUNO e PAGAMENTO	1:N	Um pagamento é de (1,1) aluno; um aluno tem (0,n) pagamentos.
registra	ALUNO e FREQUENCIA	1:N	Uma entrada é de (1,1) aluno; um aluno tem (0,n) entradas.
possui	ALUNO e FICHA_TREINO	1:N	Uma ficha é de (1,1) aluno; um aluno tem (0,n) fichas.
monta	INSTRUTOR e FICHA_TREINO	1:N	Uma ficha é montada por (1,1) instrutor; um instrutor monta (0,n) fichas.
contem	FICHA_TREINO e EXERCICIO	N:N	Uma ficha tem (1,n) exercícios; um exercício está em (0,n) fichas. Tem os atributos series, repeticoes e carga_kg.

Fonte: Elaborado pelos autores (2026).

3 MODELAGEM LÓGICA

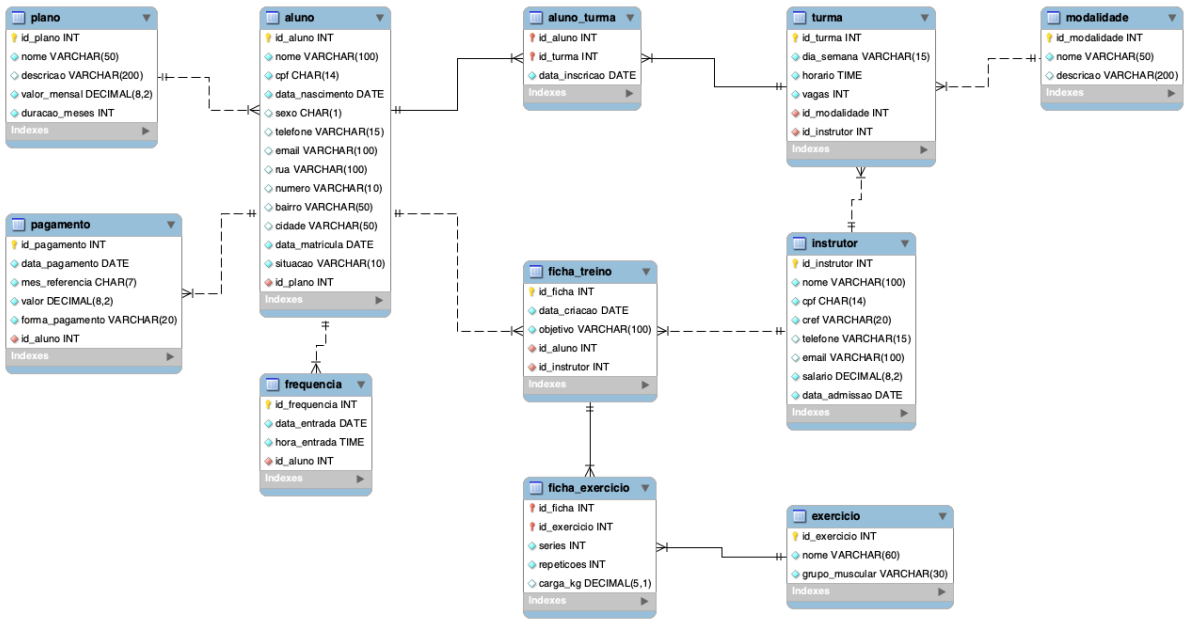
Na modelagem lógica o MER é transformado no modelo relacional, ou seja, em tabelas com colunas, chaves primárias e chaves estrangeiras. Aqui já se pensa em qual SGBD vai ser usado, no nosso caso o MySQL, por isso já aparecem os tipos de dados.

3.1 Diagrama Entidade-Relacionamento (DER)

O DER foi feito no MySQL Workbench, na parte de Data Modeling. Usamos a opção **File > Import > Reverse Engineer MySQL Create Script**, que lê o nosso script de criação das tabelas e monta o diagrama sozinho, e depois organizamos as tabelas na tela para as linhas não ficarem cruzando. O modelo foi salvo no arquivo `DER_Academia_BoaForma.mwb`.

No DER, a chave amarela marca a chave primária, o losango vermelho marca a chave estrangeira e as linhas usam a notação "pé de galinha": o tracinho duplo indica "um e somente um" e o pé de galinha indica "muitos". A linha cheia aparece quando a chave estrangeira também faz parte da chave primária (relacionamento identificador), como nas tabelas associativas, e a linha tracejada aparece nos outros casos.

Figura 2 – Diagrama Entidade-Relacionamento (DER) feito no MySQL Workbench



Fonte: Elaborado pelos autores (2026).

O banco ficou com 11 tabelas: as nove entidades do MER viraram tabelas e os dois relacionamentos N:N (participa e contem) viraram as tabelas associativas `aluno_turma` e `ficha_exercicio`. O Quadro 2 mostra as chaves de cada tabela.

Quadro 2 – Tabelas do modelo lógico e suas chaves

Tabela	Chave primária (PK)	Chaves estrangeiras (FK)	Veio de
plano	id_plano	–	Entidade PLANO
aluno	id_aluno	id_plano → plano	Entidade ALUNO + relacionamento assina
instrutor	id_instrutor	–	Entidade INSTRUTOR
modalidade	id_modalidade	–	Entidade MODALIDADE
turma	id_turma	id_modalidade → modalidade; id_instrutor → instrutor	Entidade TURMA + oferece + ministra
aluno_turma	id_aluno + id_turma	id_aluno → aluno; id_turma → turma	Relacionamento N:N participa
pagamento	id_pagamento	id_aluno → aluno	Entidade PAGAMENTO + realiza
frequencia	id_frequencia	id_aluno → aluno	Entidade FREQUENCIA + registra
ficha_treino	id_ficha	id_aluno → aluno; id_instrutor → instrutor	Entidade FICHA_TREINO + possui + monta
exercicio	id_exercicio	–	Entidade EXERCICIO
ficha_exercicio	id_ficha + id_exercicio	id_ficha → ficha_treino; id_exercicio → exercicio	Relacionamento N:N contem

Fonte: Elaborado pelos autores (2026).

3.2 Etapas da iteração dentro da modelagem

O modelo não saiu certo de primeira. Fomos fazendo e refazendo algumas vezes, e a cada volta encontrávamos alguma coisa para mudar. Abaixo explicamos cada etapa e o motivo de cada decisão.

Etapa 1 – Primeira versão. No primeiro rascunho pensamos só em quatro coisas: ALUNO, INSTRUTOR, PAGAMENTO e AULA. O plano do aluno era só um atributo dentro de ALUNO (nome do plano e valor). Quando fomos colocar os dados de exemplo, percebemos que o nome "Mensal" e o valor R$ 119,90 iam se repetir em todos os alunos desse plano, e se o preço mudasse teríamos que alterar todos eles. Por isso o plano virou a entidade **PLANO**, ligada ao aluno pelo relacionamento **assina** (N:1). Na passagem para o lógico, esse relacionamento 1:N virou a chave estrangeira `id_plano` dentro da tabela aluno, porque a regra é que a chave vai para o lado N.

Etapa 2 – Endereço. No exemplo do zoológico passado em aula, o endereço virou uma entidade separada junto com a cidade. Discutimos fazer igual, mas na

academia só o aluno tem endereço e cada aluno tem um endereço só, então decidimos deixar como atributos simples dentro de ALUNO (rua, numero, bairro e cidade). Assim evitamos dois JOINs a mais em toda consulta de aluno. Sabemos que, se um dia a academia quiser relatório por cidade sem repetir nome de cidade, dá para separar em uma tabela CIDADE.

Etapa 3 – Aula virou Modalidade e Turma. A entidade AULA tinha o nome da modalidade, o dia, o horário e o instrutor. Só que o Spinning acontece na segunda e na quarta, então o nome e a descrição da modalidade ficavam repetidos. Separamos em **MODALIDADE** (o que é a aula) e **TURMA** (quando ela acontece, quantas vagas tem e quem dá). A turma ficou com duas chaves estrangeiras: `id_modalidade` e `id_instrutor`.

Etapa 4 – Aluno e Turma (N:N). Um aluno faz várias turmas e uma turma tem vários alunos, então o relacionamento **participa** é N:N. No modelo relacional não dá para colocar a chave estrangeira em nenhum dos lados, então ele virou a tabela associativa **aluno_turma**, com chave primária composta (`id_aluno`, `id_turma`). O atributo `data_inscricao` foi para essa tabela porque ele depende do aluno e da turma ao mesmo tempo. A regra do limite de vagas não dá para mostrar só com cardinalidade, por isso ela foi resolvida depois com um trigger (seção 5.3).

Etapa 5 – Exercícios da ficha. No começo colocamos "exercícios" como um atributo multivalorado da ficha. Mas cada exercício da ficha precisa de séries, repetições e carga, e o mesmo exercício aparece em fichas diferentes. Então criamos a entidade **EXERCICIO** e o relacionamento N:N **contem**, com os atributos series, repeticoes e carga_kg. No lógico isso virou a tabela associativa **ficha_exercicio**. Colocamos a cardinalidade (1,n) do lado de EXERCICIO porque não faz sentido uma ficha sem nenhum exercício.

Etapa 6 – Frequência. Pensamos em guardar só a "última entrada" como atributo do aluno, mas aí perderíamos o histórico das outras vezes que ele foi treinar. Por isso **FREQUENCIA** virou uma entidade com data e hora, ligada ao aluno (1:N).

Etapa 7 – Pagamento. Colocamos o `mes_referencia` separado da `data_pagamento` porque o aluno pode pagar atrasado ou adiantado (por exemplo, pagar setembro no dia 2 de outubro). Também deixamos o `valor` dentro do

pagamento, e não só no plano, porque se o preço do plano mudar o valor que o aluno pagou no passado não pode mudar junto.

Etapa 8 – Situação do aluno. Em vez de apagar o aluno que cancela, criamos o atributo `situacao` (ATIVO ou INATIVO). Assim os pagamentos e a frequência antigos continuam ligados a ele. Isso também deu a ideia do trigger que reativa o aluno quando ele volta a pagar.

Etapa 9 – Conversão para o modelo lógico. Depois que o MER ficou pronto, aplicamos as regras de conversão vistas em aula e no livro do Heuser (2009):

- cada entidade virou uma tabela e cada atributo virou uma coluna;
- o atributo identificador (bolinha preta) virou a chave primária, com AUTO_INCREMENT;
- nos relacionamentos 1:N, a chave primária do lado 1 foi como chave estrangeira para a tabela do lado N;
- os relacionamentos N:N viraram tabelas associativas com chave primária composta, levando junto os atributos do relacionamento;
- escolhemos os tipos de dados: INT para códigos e quantidades, VARCHAR para textos de tamanho variável, CHAR para textos de tamanho fixo (CPF e mês de referência), DECIMAL para dinheiro e carga, DATE para datas e TIME para horários.

Etapa 10 – Conferência das formas normais. Por último conferimos o modelo com as três primeiras formas normais (ELMASRI; NAVATHE, 2018). Está na 1FN porque não sobrou nenhum atributo multivalorado nem composto nas tabelas. Está na 2FN porque, nas tabelas com chave composta (`aluno_turma` e `ficha_exercicio`), as outras colunas dependem da chave inteira e não só de uma parte dela. E está na 3FN porque tiramos as dependências entre colunas que não são chave, como o nome e o valor do plano que antes estavam dentro do aluno.

4 MODELAGEM FÍSICA

4.1 Script SQL com explicação do código

A modelagem física é o script SQL que cria o banco de verdade no MySQL. Os comandos que criam a estrutura (CREATE, ALTER, DROP) fazem parte da DDL (Data Definition Language), e os comandos que mexem nos dados (INSERT, UPDATE, DELETE) fazem parte da DML (Data Manipulation Language) (PROGRAMADORES DEPRÊ, 2026). O script foi rodado no MySQL Workbench, na parte de SQL Development, e foi dividido em três arquivos:

- `01_criacao_tabelas.sql`: cria o banco e as 11 tabelas (DDL);
- `02_insercao_dados.sql`: insere os dados de teste (DML);
- `03_consultas.sql`: as consultas, os triggers e as procedures (seção 5).

O Quadro 3 resume os comandos e as palavras-chave usadas na criação das tabelas.

Quadro 3 – Comandos SQL usados na modelagem física

Comando	Para que serve
DROP DATABASE IF EXISTS	Apaga o banco se ele já existir, para poder rodar o script várias vezes começando do zero.
CREATE DATABASE	Cria o banco academia_boaforma. Usamos o conjunto de caracteres utf8mb4 para aceitar acentos.
USE	Diz qual banco vai ser usado nos próximos comandos.
CREATE TABLE	Cria uma tabela com as suas colunas e regras.
INT, VARCHAR, CHAR, DECIMAL, DATE, TIME	Tipos de dados das colunas (número inteiro, texto, texto fixo, número com casas decimais, data e hora).
NOT NULL	Obriga a coluna a ter valor.
AUTO_INCREMENT	O MySQL gera o número da chave sozinho (1, 2, 3...).
PRIMARY KEY	Define a chave primária, que identifica cada linha e não pode repetir.
FOREIGN KEY ... REFERENCES	Define a chave estrangeira, que liga a tabela com outra e não deixa cadastrar um código que não existe.
UNIQUE	Não deixa repetir valor na coluna (usamos no CPF e no CREF).
DEFAULT	Valor padrão quando nada é informado (a situação do aluno começa como 'ATIVO').
INSERT INTO ... VALUES	Insere linhas na tabela. Dá para inserir várias de uma vez separando por vírgula.

Fonte: Elaborado pelos autores (2026).

A seguir mostramos as partes mais importantes do script. O script completo de criação está no Apêndice A.

Criação do banco

Primeiro apagamos o banco se ele já existir, criamos de novo e selecionamos ele com o USE:

```
DROP DATABASE IF EXISTS academia_boaforma;

-- Cria o banco de dados da academia
CREATE DATABASE academia_boaforma
    DEFAULT CHARACTER SET utf8mb4
    DEFAULT COLLATE utf8mb4_general_ci;

-- Fala pro MySQL que daqui pra frente vamos usar esse banco
USE academia_boaforma;
```

Tabela plano

É uma tabela simples. O `id_plano` é a chave primária com AUTO_INCREMENT e o `valor_mensal` usa DECIMAL(8,2), que guarda até 8 dígitos sendo 2 depois da vírgula, ideal para dinheiro.

```
CREATE TABLE plano (
    id_plano       INT           NOT NULL AUTO_INCREMENT, -- numero gerado sozinho
    nome           VARCHAR(50)   NOT NULL,
    descricao      VARCHAR(200),
    valor_mensal   DECIMAL(8,2)  NOT NULL,                -- 8 digitos, 2 depois da virgula
    duracao_meses  INT           NOT NULL,
    PRIMARY KEY (id_plano)
);
```

Tabela aluno

A tabela aluno tem o CPF como UNIQUE, a situação com DEFAULT 'ATIVO' e a chave estrangeira `id_plano`, que veio do relacionamento "assina". Por causa da FOREIGN KEY, o MySQL não deixa cadastrar um aluno com um plano que não existe e também não deixa apagar um plano que tem alunos.

```
CREATE TABLE aluno (
    id_aluno         INT           NOT NULL AUTO_INCREMENT,
    nome             VARCHAR(100)  NOT NULL,
    cpf              CHAR(14)      NOT NULL,             -- formato 000.000.000-00
    data_nascimento  DATE          NOT NULL,
    sexo             CHAR(1),                            -- 'M' ou 'F'
    telefone         VARCHAR(15),
    email            VARCHAR(100),
    rua              VARCHAR(100),
    numero           VARCHAR(10),
    bairro           VARCHAR(50),
    cidade           VARCHAR(50),
    data_matricula   DATE          NOT NULL,
    situacao         VARCHAR(10)   NOT NULL DEFAULT 'ATIVO', -- ATIVO ou INATIVO
    id_plano         INT           NOT NULL,
    PRIMARY KEY (id_aluno),
    UNIQUE (cpf),                                        -- nao pode ter dois alunos com o
mesmo CPF
    FOREIGN KEY (id_plano) REFERENCES plano (id_plano)
);
```

Tabela turma

A turma tem duas chaves estrangeiras, uma para a modalidade e outra para o instrutor, e a coluna `vagas`, que é usada pelo trigger que bloqueia inscrição em turma lotada.

```
CREATE TABLE turma (
    id_turma       INT          NOT NULL AUTO_INCREMENT,
    dia_semana     VARCHAR(15)  NOT NULL,
    horario        TIME         NOT NULL,
    vagas          INT          NOT NULL,
    id_modalidade  INT          NOT NULL,
    id_instrutor   INT          NOT NULL,
    PRIMARY KEY (id_turma),
    FOREIGN KEY (id_modalidade) REFERENCES modalidade (id_modalidade),
    FOREIGN KEY (id_instrutor)  REFERENCES instrutor (id_instrutor)
);
```

Tabela associativa aluno_turma

Essa tabela veio do relacionamento N:N "participa". A chave primária é composta pelas duas chaves estrangeiras, então o mesmo aluno não consegue se inscrever duas vezes na mesma turma. A tabela `ficha_exercicio` segue a mesma ideia.

```
CREATE TABLE aluno_turma (
    id_aluno        INT   NOT NULL,
    id_turma        INT   NOT NULL,
    data_inscricao  DATE  NOT NULL,
    PRIMARY KEY (id_aluno, id_turma),
    FOREIGN KEY (id_aluno) REFERENCES aluno (id_aluno),
    FOREIGN KEY (id_turma) REFERENCES turma (id_turma)
);
```

Inserção dos dados de teste

Para testar as consultas, inserimos dados inventados (nomes, CPFs e telefones são fictícios). Usamos um INSERT com várias linhas de uma vez. Não informamos o id porque ele é gerado pelo AUTO_INCREMENT. Exemplo com os planos:

```
INSERT INTO plano (nome, descricao, valor_mensal, duracao_meses) VALUES
('Mensal',     'Acesso livre a musculacao',                  119.90, 1),
('Trimestral', 'Musculacao + 1 modalidade',                  109.90, 3),
('Semestral',  'Musculacao + todas as modalidades',           99.90, 6),
('Anual',      'Acesso total a academia + avaliacao fisica',  89.90, 12);
```

A ordem dos INSERTs importa: primeiro entram as tabelas que não dependem de ninguém (plano, instrutor, modalidade, exercicio) e depois as que têm chave estrangeira. Se tentar inserir um aluno antes do plano, o MySQL dá erro de chave estrangeira. O Quadro 4 mostra quantos registros ficaram em cada tabela depois do script 02.

Quadro 4 – Quantidade de registros de teste por tabela

Tabela	Registros	Tabela	Registros
plano	4	pagamento	43
aluno	15	ficha_treino	7
instrutor	5	exercicio	14
modalidade	6	ficha_exercicio	31
turma	8	frequencia	37
aluno_turma	25	Total	195

Fonte: Elaborado pelos autores (2026).

5 CONSULTAS SQL

As consultas estão no arquivo `03_consultas.sql`. Ao todo fizemos **31 consultas numeradas**: 5 básicas, 21 intermediárias e 5 testes das consultas avançadas, além da criação de 2 triggers e 3 stored procedures. Para cada consulta mostramos o objetivo, o código, uma explicação e o resultado que apareceu no MySQL. Quando o resultado tinha muitas linhas ou colunas, mostramos só o começo. O Quadro 5 mostra onde está cada comando pedido no trabalho.

Quadro 5 – Onde cada comando pedido aparece

Comando / recurso	Consultas
SELECT e FROM	01 a 05 (e todas as outras)
DISTINCT	06
Colunas calculadas (+, -, *, /)	07, 25
WHERE com operadores de comparação	08 (>=), 11 (<>), 18 (>), 27 e 31 (=)
AND, OR e NOT	09
SUM, AVG, MAX e MIN	10
COUNT	11, 16, 17, 25, 26
LIKE	12, 24
BETWEEN	13, 26
IN / NOT IN	14, 19
ORDER BY	15, 16, 20, 21, 23, 25, 26
GROUP BY	16, 17, 25, 26
HAVING	17
Subconsultas	18, 19, procedure 3
JOIN (INNER e LEFT)	17, 20, 21, 22, 24, 25, 26
UNION	23
Trigger	triggers 1 e 2; testes 27 e 31
Stored procedure	procedures 1, 2 e 3; testes 28, 29 e 30

Fonte: Elaborado pelos autores (2026).

5.1 Consultas básicas

As consultas básicas usam só o SELECT (o que mostrar) e o FROM (de qual tabela). Segundo o W3Schools (2026), o SELECT é o comando usado para buscar dados no banco, e o resultado fica em uma tabela chamada result-set.

*Consulta 01 – SELECT * (todas as colunas)*

Objetivo: Mostrar todos os dados de todos os alunos cadastrados.

```
SELECT * FROM aluno;
```

Explicação: O asterisco (*) pede todas as colunas da tabela. É o SELECT mais simples que existe.

Resultado:

id_aluno	nome	cpf	data_nascimento	sexo	telefone
1	Ana Beatriz Santos	201.302.403-01	2001-04-12	F	(21) 99701-0001
2	Bruno Henrique Oliveira	201.302.403-02	1998-09-23	M	(21) 99701-0002
3	Camila Ferreira Gomes	201.302.403-03	2003-01-30	F	(21) 99701-0003
4	Diego Martins Ribeiro	201.302.403-04	1995-11-02	M	(21) 99701-0004
5	Eduarda Lima Barbosa	201.302.403-05	2005-06-18	F	(21) 99701-0005
6	Felipe Araújo Cardoso	201.302.403-06	1990-03-07	M	(21) 99701-0006

15 linha(s) retornada(s) (mostrando só as 6 primeiras colunas de 14; mostrando 6 de 15 linhas).

Consulta 02 – SELECT com colunas escolhidas

Objetivo: Mostrar só o nome, o telefone e o e-mail dos alunos.

```
SELECT nome, telefone, email FROM aluno;
```

Explicação: Em vez do *, escrevemos o nome das colunas que queremos ver, separadas por vírgula.

Resultado:

nome	telefone	email
Ana Beatriz Santos	(21) 99701-0001	ana.santos@gmail.com
Bruno Henrique Oliveira	(21) 99701-0002	bruno.oliveira@hotmail.com
Camila Ferreira Gomes	(21) 99701-0003	camila.gomes@gmail.com
Diego Martins Ribeiro	(21) 99701-0004	diego.ribeiro@yahoo.com
Eduarda Lima Barbosa	(21) 99701-0005	duda.barbosa@gmail.com
Felipe Araújo Cardoso	(21) 99701-0006	felipe.cardoso@outlook.com

15 linha(s) retornada(s) (mostrando 6 de 15 linhas).

Consulta 03 – Listar os planos

Objetivo: Ver todos os planos que a academia vende.

```
SELECT * FROM plano;
```

Explicação: Consulta básica na tabela plano.

Resultado:

id_plano	nome	descricao	valor_mensal	duracao_meses
1	Mensal	Acesso livre a musculacao	119.90	1
2	Trimestral	Musculacao + 1 modalidade	109.90	3
3	Semestral	Musculacao + todas as modalidades	99.90	6
4	Anual	Acesso total a academia + avaliacao fisica	89.90	12

4 linha(s) retornada(s).

Consulta 04 – Dados dos instrutores

Objetivo: Mostrar o nome, o CREF e o salário dos instrutores.

```
SELECT nome, cref, salario FROM instrutor;
```

Explicação: Consulta básica escolhendo três colunas da tabela instrutor.

Resultado:

nome	cref	salario
Carlos Eduardo Lima	012345-G/RJ	3500.00
Fernanda Souza Alves	023456-G/RJ	3200.00
Ricardo Mendes	034567-G/RJ	2800.00
Juliana Rocha Costa	045678-G/RJ	3000.00
Thiago Pereira Nunes	056789-G/RJ	2600.00

5 linha(s) retornada(s).

Consulta 05 – Lista de exercícios

Objetivo: Mostrar os exercícios e o grupo muscular de cada um.

```
SELECT nome, grupo_muscular FROM exercicio;
```

Explicação: Consulta básica na tabela exercicio.

Resultado:

nome	grupo_muscular
Supino reto	Peito
Crucifixo	Peito
Puxada frontal	Costas
Remada baixa	Costas
Agachamento livre	Pernas
Leg press 45	Pernas

14 linha(s) retornada(s) (mostrando 6 de 14 linhas).

5.2 Consultas intermediárias

Aqui usamos filtros (WHERE), operadores, funções de agregação, agrupamento, ordenação, subconsultas, junção de tabelas (JOIN) e UNION.

Consulta 06 – DISTINCT

Objetivo: Descobrir quais formas de pagamento já foram usadas.

```
SELECT DISTINCT forma_pagamento FROM pagamento;
```

Explicação: Existem 43 pagamentos, mas só 4 formas diferentes. O DISTINCT tira as linhas repetidas do resultado.

Resultado:

forma_pagamento
PIX

forma_pagamento
CARTAO DE CREDITO
CARTAO DE DEBITO
DINHEIRO

4 linha(s) retornada(s).

*Consulta 07 – Colunas calculadas (+, -, *, /)*

Objetivo: Calcular o valor total de cada plano, o total com a taxa de matrícula de R$ 50,00, a mensalidade com 10% de desconto e quanto o plano custa por dia.

```
SELECT nome,
       valor_mensal,
       valor_mensal * duracao_meses           AS valor_total_plano,
       valor_mensal * duracao_meses + 50      AS total_com_matricula,
       ROUND(valor_mensal - (valor_mensal * 0.10), 2) AS mensal_com_desconto,
       ROUND(valor_mensal / 30, 2)            AS valor_por_dia
FROM plano;
```

Explicação: Dá para fazer contas direto no SELECT usando +, -, * e /. O AS dá um nome (apelido) para a coluna calculada e o ROUND arredonda o valor por dia para 2 casas decimais.

Resultado:

nome	valor_mensal	valor_total_plano	total_com_matricula	mensal_com_desconto	valor_por_dia
Mensal	119.90	119.90	169.90	107.91	4.00
Trimestral	109.90	329.70	379.70	98.91	3.66
Semestral	99.90	599.40	649.40	89.91	3.33
Anual	89.90	1078.80	1128.80	80.91	3.00

4 linha(s) retornada(s).

Consulta 08 – WHERE com operador de comparação (>=)

Objetivo: Listar os instrutores que ganham R$ 3.000,00 ou mais.

```
SELECT nome, salario
FROM instrutor
WHERE salario >= 3000;
```

Explicação: O WHERE filtra as linhas. Os operadores de comparação são =, <>, >, <, >= e <=.

Resultado:

nome	salario
Carlos Eduardo Lima	3500.00
Fernanda Souza Alves	3200.00
Juliana Rocha Costa	3000.00

3 linha(s) retornada(s).

Consulta 09 – Operadores lógicos AND, OR e NOT

Objetivo: Listar os alunos que não estão inativos e moram em Niterói ou São Gonçalo.

```
SELECT nome, cidade, situacao
FROM aluno
WHERE NOT situacao = 'INATIVO'
  AND (cidade = 'Niterói' OR cidade = 'São Gonçalo');
```

Explicação: O NOT inverte a condição, o AND exige as duas condições e o OR aceita qualquer uma. Os parênteses são importantes, senão o AND é feito antes do OR.

Resultado:

nome	cidade	situacao
Bruno Henrique Oliveira	Niterói	ATIVO
Diego Martins Ribeiro	São Gonçalo	ATIVO
Felipe Araújo Cardoso	Niterói	ATIVO
Larissa Monteiro Dias	Niterói	ATIVO

4 linha(s) retornada(s).

Consulta 10 – Funções de agregação SUM, AVG, MAX e MIN

Objetivo: Fazer um resumo de todos os pagamentos recebidos.

```
SELECT SUM(valor)           AS total_recebido,
       ROUND(AVG(valor), 2) AS media_por_pagamento,
       MAX(valor)           AS maior_pagamento,
       MIN(valor)           AS menor_pagamento
FROM pagamento;
```

Explicação: As funções de agregação juntam várias linhas em um único valor: soma, média, maior e menor. O ROUND arredonda a média para 2 casas.

Resultado:

total_recebido	media_por_pagamento	maior_pagamento	menor_pagamento
4395.70	102.23	119.90	89.90

1 linha(s) retornada(s).

Consulta 11 – COUNT e operador <>

Objetivo: Contar quantos alunos estão ativos.

```
SELECT COUNT(*) AS total_alunos_ativos
FROM aluno
WHERE situacao <> 'INATIVO';
```

Explicação: O COUNT(*) conta as linhas que passaram pelo WHERE. O operador <> significa "diferente de", então a consulta conta todo mundo que não está INATIVO.

Resultado:

total_alunos_ativos
13

1 linha(s) retornada(s).

Consulta 12 – LIKE

Objetivo: Encontrar os alunos que usam e-mail do Gmail.

```
SELECT nome, email
FROM aluno
WHERE email LIKE '%@gmail.com';
```

Explicação: O LIKE compara textos usando curingas. O % significa "qualquer coisa", então '%@gmail.com' pega tudo que termina com @gmail.com.

Resultado:

nome	email
Ana Beatriz Santos	ana.santos@gmail.com
Camila Ferreira Gomes	camila.gomes@gmail.com
Eduarda Lima Barbosa	duda.barbosa@gmail.com
Gabriela Nascimento Reis	gabi.reis@gmail.com
Henrique Costa Moreira	henrique.moreira@gmail.com
Isabela Teixeira Pinto	isa.pinto@gmail.com

11 linha(s) retornada(s) (mostrando 6 de 11 linhas).

Consulta 13 – BETWEEN

Objetivo: Listar os alunos que nasceram entre 2000 e 2005.

```
SELECT nome, data_nascimento
FROM aluno
WHERE data_nascimento BETWEEN '2000-01-01' AND '2005-12-31';
```

Explicação: O BETWEEN pega os valores dentro de um intervalo, incluindo as duas pontas.

Resultado:

nome	data_nascimento
Ana Beatriz Santos	2001-04-12
Camila Ferreira Gomes	2003-01-30
Eduarda Lima Barbosa	2005-06-18
Isabela Teixeira Pinto	2002-02-09
João Pedro Almeida	2004-07-21
Marcos Vinícius Rocha	2000-10-11

6 linha(s) retornada(s).

Consulta 14 – IN

Objetivo: Mostrar os exercícios de peito, costas ou ombros.

```
SELECT nome, grupo_muscular
FROM exercicio
WHERE grupo_muscular IN ('Peito', 'Costas', 'Ombros');
```

Explicação: O IN é um jeito mais curto de escrever vários OR para a mesma coluna.

Resultado:

nome	grupo_muscular
Supino reto	Peito

nome	grupo_muscular
Crucifixo	Peito
Puxada frontal	Costas
Remada baixa	Costas
Desenvolvimento com halteres	Ombros
Elevação lateral	Ombros

6 linha(s) retornada(s).

Consulta 15 – ORDER BY

Objetivo: Ordenar os alunos do que entrou mais recentemente para o mais antigo.

```
SELECT nome, data_matricula
FROM aluno
ORDER BY data_matricula DESC;
```

Explicação: O ORDER BY ordena o resultado. ASC é crescente (padrão) e DESC é decrescente.

Resultado:

nome	data_matricula
Vitória Gonçalves Freitas	2026-10-20
Isabela Teixeira Pinto	2026-10-01
Eduarda Lima Barbosa	2026-09-03
Larissa Monteiro Dias	2026-08-20
Bruno Henrique Oliveira	2026-08-01
Diego Martins Ribeiro	2026-07-14

15 linha(s) retornada(s) (mostrando 6 de 15 linhas).

Consulta 16 – GROUP BY

Objetivo: Saber quanto foi recebido em cada forma de pagamento.

```
SELECT forma_pagamento,
       COUNT(*)   AS qtd_pagamentos,
       SUM(valor) AS total
FROM pagamento
GROUP BY forma_pagamento
ORDER BY total DESC;
```

Explicação: O GROUP BY junta as linhas que têm o mesmo valor na coluna e aí as funções de agregação são calculadas para cada grupo.

Resultado:

forma_pagamento	qtd_pagamentos	total
PIX	20	2028.00
CARTAO DE CREDITO	11	1138.90
CARTAO DE DEBITO	8	759.20
DINHEIRO	4	469.60

4 linha(s) retornada(s).

Consulta 17 – GROUP BY com HAVING

Objetivo: Descobrir quais planos têm 4 alunos ou mais.

```
SELECT p.nome AS plano,
       COUNT(a.id_aluno) AS qtd_alunos
FROM plano p
INNER JOIN aluno a ON a.id_plano = p.id_plano
GROUP BY p.id_plano, p.nome
HAVING COUNT(a.id_aluno) >= 4;
```

Explicação: O HAVING filtra depois do agrupamento (o WHERE filtra antes). Aqui também usamos um JOIN para mostrar o nome do plano.

Resultado:

plano	qtd_alunos
Mensal	5
Anual	4

2 linha(s) retornada(s).

Consulta 18 – Subconsulta

Objetivo: Listar os instrutores que ganham mais do que a média dos salários.

```
SELECT nome, salario
FROM instrutor
WHERE salario > (SELECT AVG(salario) FROM instrutor);
```

Explicação: A consulta de dentro calcula a média (R$ 3.020,00) e a consulta de fora usa esse valor no WHERE.

Resultado:

nome	salario
Carlos Eduardo Lima	3500.00
Fernanda Souza Alves	3200.00

2 linha(s) retornada(s).

Consulta 19 – Subconsulta com NOT IN

Objetivo: Encontrar os alunos que nunca fizeram nenhum pagamento.

```
SELECT id_aluno, nome, data_matricula
FROM aluno
WHERE id_aluno NOT IN (SELECT id_aluno FROM pagamento);
```

Explicação: A subconsulta devolve a lista de alunos que já pagaram, e o NOT IN pega quem não está nessa lista.

Resultado:

id_aluno	nome	data_matricula
15	Vitória Gonçalves Freitas	2026-10-20

1 linha(s) retornada(s).

Consulta 20 – INNER JOIN

Objetivo: Mostrar cada aluno junto com o nome e o valor do plano que ele assinou.

```
SELECT a.nome AS aluno,
       p.nome AS plano,
       p.valor_mensal
FROM aluno a
INNER JOIN plano p ON a.id_plano = p.id_plano
ORDER BY a.nome;
```

Explicação: O JOIN junta duas tabelas usando a chave estrangeira (aluno.id_plano) e a chave primária (plano.id_plano). Usamos apelidos (a e p) para não ter que escrever o nome inteiro da tabela.

Resultado:

aluno	plano	valor_mensal
Ana Beatriz Santos	Anual	89.90
Bruno Henrique Oliveira	Mensal	119.90
Camila Ferreira Gomes	Semestral	99.90
Diego Martins Ribeiro	Trimestral	109.90
Eduarda Lima Barbosa	Mensal	119.90
Felipe Araújo Cardoso	Anual	89.90

15 linha(s) retornada(s) (mostrando 6 de 15 linhas).

Consulta 21 – JOIN com três tabelas

Objetivo: Montar a grade de horários das turmas com o nome da modalidade e do instrutor.

```
SELECT t.id_turma,
       m.nome AS modalidade,
       i.nome AS instrutor,
       t.dia_semana,
       t.horario,
       t.vagas
FROM turma t
INNER JOIN modalidade m ON t.id_modalidade = m.id_modalidade
INNER JOIN instrutor  i ON t.id_instrutor  = i.id_instrutor
ORDER BY m.nome, t.dia_semana;
```

Explicação: Dá para encadear vários JOINs. A tabela turma tem duas chaves estrangeiras, uma para modalidade e outra para instrutor.

Resultado:

id_turma	modalidade	instrutor	dia_semana	horario	vagas
7	Funcional	Ricardo Mendes	Sábado	09:00:00	12
5	Funcional	Ricardo Mendes	Segunda	19:00:00	12
3	Pilates	Juliana Rocha Costa	Terça	18:00:00	3
2	Spinning	Fernanda Souza Alves	Quarta	07:00:00	10
1	Spinning	Fernanda Souza Alves	Segunda	07:00:00	10
4	Yoga	Juliana Rocha Costa	Quinta	18:00:00	8

8 linha(s) retornada(s) (mostrando 6 de 8 linhas).

Consulta 22 – LEFT JOIN

Objetivo: Listar os alunos que ainda não têm ficha de treino.

```
SELECT a.nome, a.situacao
FROM aluno a
LEFT JOIN ficha_treino f ON a.id_aluno = f.id_aluno
WHERE f.id_ficha IS NULL;
```

Explicação: O LEFT JOIN traz todos os alunos, mesmo os que não têm ficha. Quando não tem ficha, as colunas da ficha ficam NULL, e é isso que o WHERE procura.

Resultado:

nome	situacao
Diego Martins Ribeiro	ATIVO
Eduarda Lima Barbosa	ATIVO
Gabriela Nascimento Reis	ATIVO
Henrique Costa Moreira	INATIVO
Isabela Teixeira Pinto	ATIVO
Marcos Vinícius Rocha	INATIVO

8 linha(s) retornada(s) (mostrando 6 de 8 linhas).

Consulta 23 – UNION

Objetivo: Fazer uma lista única com todas as pessoas da academia (alunos e instrutores).

```
SELECT nome, email, 'Aluno' AS tipo FROM aluno
UNION
SELECT nome, email, 'Instrutor' AS tipo FROM instrutor
ORDER BY nome;
```

Explicação: O UNION junta o resultado de dois SELECTs. Os dois precisam ter a mesma quantidade de colunas e tipos parecidos.

Resultado:

nome	email	tipo
Ana Beatriz Santos	ana.santos@gmail.com	Aluno
Bruno Henrique Oliveira	bruno.oliveira@hotmail.com	Aluno
Camila Ferreira Gomes	camila.gomes@gmail.com	Aluno
Carlos Eduardo Lima	carlos.lima@boaforma.com	Instrutor
Diego Martins Ribeiro	diego.ribeiro@yahoo.com	Aluno
Eduarda Lima Barbosa	duda.barbosa@gmail.com	Aluno

20 linha(s) retornada(s) (mostrando 6 de 20 linhas).

Consulta 24 – JOIN com cinco tabelas

Objetivo: Mostrar a ficha de treino completa da aluna Ana.

```
SELECT a.nome  AS aluno,
       i.nome  AS instrutor,
       f.objetivo,
       e.nome  AS exercicio,
       e.grupo_muscular,
       fe.series,
       fe.repeticoes,
       fe.carga_kg
FROM ficha_treino f
INNER JOIN aluno           a  ON f.id_aluno     = a.id_aluno
INNER JOIN instrutor       i  ON f.id_instrutor = i.id_instrutor
INNER JOIN ficha_exercicio fe ON f.id_ficha     = fe.id_ficha
INNER JOIN exercicio       e  ON fe.id_exercicio = e.id_exercicio
WHERE a.nome LIKE 'Ana%';
```

Explicação: Para montar a ficha precisamos de aluno, instrutor, ficha_treino, ficha_exercicio (tabela associativa) e exercicio.

Resultado:

aluno	instrutor	objetivo	exercicio	grupo_muscular	series
Ana Beatriz Santos	Carlos Eduardo Lima	Hipertrofia	Supino reto	Peito	4
Ana Beatriz Santos	Carlos Eduardo Lima	Hipertrofia	Puxada frontal	Costas	4
Ana Beatriz Santos	Carlos Eduardo Lima	Hipertrofia	Agachamento livre	Pernas	4
Ana Beatriz Santos	Carlos Eduardo Lima	Hipertrofia	Rosca direta	Bíceps	3
Ana Beatriz Santos	Carlos Eduardo Lima	Hipertrofia	Tríceps pulley	Tríceps	3

5 linha(s) retornada(s) (mostrando só as 6 primeiras colunas de 8).

Consulta 25 – LEFT JOIN, GROUP BY e coluna calculada

Objetivo: Ver quantos inscritos cada turma tem e quantas vagas ainda sobram.

```
SELECT t.id_turma,
       m.nome AS modalidade,
       t.dia_semana,
       t.vagas,
       COUNT(at.id_aluno)           AS inscritos,
       t.vagas - COUNT(at.id_aluno) AS vagas_livres
FROM turma t
INNER JOIN modalidade m  ON t.id_modalidade = m.id_modalidade
LEFT JOIN  aluno_turma at ON t.id_turma     = at.id_turma
GROUP BY t.id_turma, m.nome, t.dia_semana, t.vagas
ORDER BY vagas_livres;
```

Explicação: Usamos LEFT JOIN para não sumir com turma vazia, COUNT para contar os inscritos e uma subtração para calcular as vagas livres. A turma de Pilates aparece lotada (0 vagas).

Resultado:

id_turma	modalidade	dia_semana	vagas	inscritos	vagas_livres
3	Pilates	Terça	3	3	0
4	Yoga	Quinta	8	3	5
1	Spinning	Segunda	10	4	6
2	Spinning	Quarta	10	2	8
5	Funcional	Segunda	12	4	8
6	Zumba	Sexta	15	5	10

8 linha(s) retornada(s) (mostrando 6 de 8 linhas).

Consulta 26 – JOIN, WHERE e GROUP BY

Objetivo: Contar quantas vezes cada aluno foi à academia na primeira semana de novembro.

```
SELECT a.nome,
       COUNT(*) AS qtd_entradas
FROM frequencia f
INNER JOIN aluno a ON f.id_aluno = a.id_aluno
WHERE f.data_entrada BETWEEN '2026-11-01' AND '2026-11-07'
GROUP BY a.id_aluno, a.nome
ORDER BY qtd_entradas DESC;
```

Explicação: Junta frequencia com aluno, filtra as datas com BETWEEN e conta as entradas de cada aluno.

Resultado:

nome	qtd_entradas
Ana Beatriz Santos	3
Rafael Souza Batista	3
Felipe Araújo Cardoso	3
Eduarda Lima Barbosa	2
Bruno Henrique Oliveira	1
Camila Ferreira Gomes	1

13 linha(s) retornada(s) (mostrando 6 de 13 linhas).

5.3 Consultas avançadas: triggers e stored procedures

Um **trigger** (gatilho) é um bloco de código que fica guardado no banco e roda sozinho quando acontece um evento numa tabela, como um INSERT, UPDATE ou DELETE. Ele pode rodar antes (BEFORE) ou depois (AFTER) do evento. Uma **stored procedure** (procedimento armazenado) também fica guardada no banco, mas só roda quando alguém chama ela com o comando CALL, e pode receber parâmetros (ORACLE, 2026a).

Como o código dos triggers e procedures tem vários ponto e vírgula dentro do BEGIN ... END, usamos o comando `DELIMITER $$` para trocar temporariamente o fim do comando para $$. Sem isso o MySQL acha que o comando acabou no primeiro ponto e vírgula e dá erro. No final voltamos para `DELIMITER ;`. Antes de cada CREATE também colocamos um DROP ... IF EXISTS para poder rodar o script mais de uma vez.

Trigger 1: trg_pagamento_reativa_aluno

Esse trigger é disparado **depois** (AFTER INSERT) de cada pagamento inserido. A palavra `NEW` representa a linha que acabou de entrar na tabela pagamento, então `NEW.id_aluno` é o aluno que pagou. Se ele estava INATIVO, o UPDATE muda a situação para ATIVO.

```
DROP TRIGGER IF EXISTS trg_pagamento_reativa_aluno;

DELIMITER $$
CREATE TRIGGER trg_pagamento_reativa_aluno
AFTER INSERT ON pagamento
FOR EACH ROW
BEGIN
    -- NEW e a linha que acabou de ser inserida em pagamento
    UPDATE aluno
       SET situacao = 'ATIVO'
     WHERE id_aluno = NEW.id_aluno
       AND situacao = 'INATIVO';
END $$
DELIMITER ;
```

Trigger 2: trg_verifica_vagas

Esse trigger é disparado **antes** (BEFORE INSERT) de inscrever um aluno numa turma. Ele cria duas variáveis com DECLARE, conta quantos alunos já estão na turma, busca o limite de vagas e, se a turma estiver cheia, usa o `SIGNAL SQLSTATE '45000'` para gerar um erro com mensagem. Como o erro acontece antes do INSERT, a linha não é gravada. Foi o jeito que encontramos de garantir a regra das vagas, que não dá para representar só com a cardinalidade do MER.

```
DROP TRIGGER IF EXISTS trg_verifica_vagas;

DELIMITER $$
CREATE TRIGGER trg_verifica_vagas
BEFORE INSERT ON aluno_turma
FOR EACH ROW
BEGIN
    DECLARE v_inscritos INT;
    DECLARE v_vagas INT;

    -- conta quantos alunos ja estao na turma
    SELECT COUNT(*) INTO v_inscritos
      FROM aluno_turma
     WHERE id_turma = NEW.id_turma;

    -- pega o limite de vagas da turma
    SELECT vagas INTO v_vagas
      FROM turma
     WHERE id_turma = NEW.id_turma;

    -- se ja lotou, gera um erro com mensagem
    IF v_inscritos >= v_vagas THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Turma lotada! Nao ha vagas disponiveis.';
    END IF;
END $$
DELIMITER ;
```

Procedure 1: sp_cadastrar_aluno

Procedure com parâmetros de entrada (IN). Antes de inserir, ela confere com `IF EXISTS` se o CPF já está cadastrado; se estiver, gera um erro. Se não estiver, insere o aluno usando `CURDATE()` (data de hoje) como data de matrícula e depois mostra o aluno criado usando `LAST_INSERT_ID()`, que devolve o último id gerado pelo AUTO_INCREMENT.

```
DROP PROCEDURE IF EXISTS sp_cadastrar_aluno;

DELIMITER $$
CREATE PROCEDURE sp_cadastrar_aluno(
    IN p_nome        VARCHAR(100),
    IN p_cpf         CHAR(14),
    IN p_nascimento  DATE,
    IN p_sexo        CHAR(1),
    IN p_telefone    VARCHAR(15),
    IN p_email       VARCHAR(100),
    IN p_id_plano    INT
)
BEGIN
    IF EXISTS (SELECT 1 FROM aluno WHERE cpf = p_cpf) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Ja existe um aluno com esse CPF.';
    ELSE
        INSERT INTO aluno (nome, cpf, data_nascimento, sexo, telefone, email,
                           data_matricula, situacao, id_plano)
        VALUES (p_nome, p_cpf, p_nascimento, p_sexo, p_telefone, p_email,
                CURDATE(), 'ATIVO', p_id_plano);

        -- mostra o aluno que acabou de ser cadastrado
        -- LAST_INSERT_ID() devolve o id gerado pelo AUTO_INCREMENT
        SELECT * FROM aluno WHERE id_aluno = LAST_INSERT_ID();
    END IF;
END $$
DELIMITER ;
```

Procedure 2: sp_relatorio_mensal

Procedure que recebe o mês (no formato AAAA-MM) e mostra quantos pagamentos entraram e o total recebido, separado por forma de pagamento. É uma consulta com GROUP BY "guardada" no banco para ser usada sempre que precisar.

```
DROP PROCEDURE IF EXISTS sp_relatorio_mensal;

DELIMITER $$
CREATE PROCEDURE sp_relatorio_mensal(IN p_mes CHAR(7))
BEGIN
    SELECT p_mes             AS mes,
           forma_pagamento,
           COUNT(*)          AS qtd_pagamentos,
           SUM(valor)        AS total_recebido
      FROM pagamento
     WHERE mes_referencia = p_mes
     GROUP BY forma_pagamento
     ORDER BY total_recebido DESC;
END $$
DELIMITER ;
```

Procedure 3: sp_alunos_pendentes

Procedure que recebe o mês e lista os alunos ativos que ainda não pagaram aquele mês. Ela usa JOIN com plano (para mostrar o valor devido) e uma subconsulta com NOT IN.

```
DROP PROCEDURE IF EXISTS sp_alunos_pendentes;

DELIMITER $$
CREATE PROCEDURE sp_alunos_pendentes(IN p_mes CHAR(7))
BEGIN
    SELECT a.id_aluno,
           a.nome,
           a.telefone,
           p.nome         AS plano,
           p.valor_mensal AS valor_devido
      FROM aluno a
     INNER JOIN plano p ON a.id_plano = p.id_plano
     WHERE a.situacao = 'ATIVO'
       AND a.id_aluno NOT IN (SELECT id_aluno
                              FROM pagamento
                             WHERE mes_referencia = p_mes)
     ORDER BY a.nome;
END $$
DELIMITER ;
```

Testando os triggers e as procedures

Depois de criar os triggers e as procedures, fizemos os testes abaixo para mostrar que eles funcionam.

Consulta 27 – Teste do TRIGGER 1

Objetivo: Mostrar que um aluno inativo volta a ficar ativo sozinho quando paga.

```
SELECT id_aluno, nome, situacao FROM aluno WHERE id_aluno = 12;
-- ...ele paga novembro...
INSERT INTO pagamento (data_pagamento, mes_referencia, valor, forma_pagamento, id_aluno)
VALUES ('2026-11-08', '2026-11', 119.90, 'PIX', 12);
-- ...e o trigger deixa ele ATIVO de novo sozinho
SELECT id_aluno, nome, situacao FROM aluno WHERE id_aluno = 12;
```

Explicação: O Marcos estava INATIVO. Depois do INSERT na tabela pagamento, o trigger fez o UPDATE e ele ficou ATIVO, sem ninguém mexer na tabela aluno.

Resultado:

Antes do pagamento:

id_aluno	nome	situacao
12	Marcos Vinícius Rocha	INATIVO

1 linha(s) retornada(s).

Depois do pagamento:

id_aluno	nome	situacao
12	Marcos Vinícius Rocha	ATIVO

1 linha(s) retornada(s).

Consulta 28 – Teste da PROCEDURE 1

Objetivo: Cadastrar um aluno novo chamando a procedure.

```
CALL sp_cadastrar_aluno('Lucas Andrade Silva', '201.302.403-16', '2003-08-19',
                        'M', '(21) 99701-0016', 'lucas.silva@gmail.com', 2);
```

Explicação: O CALL executa a procedure. Ela inseriu o aluno com a data de hoje como matrícula, situação ATIVO, e já mostrou o registro criado.

Resultado:

id_aluno	nome	cpf	data_nascimento	sexo	telefone
16	Lucas Andrade Silva	201.302.403-16	2003-08-19	M	(21) 99701-0016

1 linha(s) retornada(s) (mostrando só as 6 primeiras colunas de 14).

Consulta 29 – Teste da PROCEDURE 2

Objetivo: Gerar o relatório de recebimentos de outubro de 2026.

```
CALL sp_relatorio_mensal('2026-10');
```

Explicação: A procedure recebe o mês como parâmetro, então serve para qualquer mês sem precisar reescrever a consulta.

Resultado:

mes	forma_pagamento	qtd_pagamentos	total_recebido
2026-10	PIX	6	619.40
2026-10	CARTAO DE CREDITO	3	309.70
2026-10	CARTAO DE DEBITO	2	189.80
2026-10	DINHEIRO	1	119.90

4 linha(s) retornada(s).

Consulta 30 – Teste da PROCEDURE 3

Objetivo: Listar quem ainda não pagou novembro de 2026.

```
CALL sp_alunos_pendentes('2026-11');
```

Explicação: Aparecem os alunos ativos sem pagamento em 2026-11, incluindo o Lucas que acabou de ser cadastrado pela procedure 1.

Resultado:

id_aluno	nome	telefone	plano	valor_devido
2	Bruno Henrique Oliveira	(21) 99701-0002	Mensal	119.90
4	Diego Martins Ribeiro	(21) 99701-0004	Trimestral	109.90
7	Gabriela Nascimento Reis	(21) 99701-0007	Semestral	99.90
16	Lucas Andrade Silva	(21) 99701-0016	Trimestral	109.90
13	Mariana Cavalcanti	(21) 99701-0013	Semestral	99.90
15	Vitória Gonçalves Freitas	(21) 99701-0015	Mensal	119.90

6 linha(s) retornada(s).

Consulta 31 – Teste do TRIGGER 2

Objetivo: Tentar colocar mais uma aluna na turma de Pilates, que já está lotada.

```
SELECT t.id_turma, t.vagas, COUNT(at.id_aluno) AS inscritos
FROM turma t
LEFT JOIN aluno_turma at ON t.id_turma = at.id_turma
WHERE t.id_turma = 3
GROUP BY t.id_turma, t.vagas;


-- agora tenta colocar a Eduarda (id 5) no Pilates.
-- ATENCAO: esse comando VAI DAR ERRO DE PROPOSITO!
-- A mensagem esperada e: "Turma lotada! Nao ha vagas disponiveis."
-- (por isso ele ficou por ultimo no arquivo)
INSERT INTO aluno_turma (id_aluno, id_turma, data_inscricao)
VALUES (5, 3, '2026-11-08');
```

Explicação: Primeiro o SELECT mostra que a turma 3 tem 3 vagas e 3 inscritos. Depois o INSERT é bloqueado pelo trigger com a mensagem de erro que nós definimos. Esse erro é proposital.

Resultado:

id_turma	vagas	inscritos
3	3	3

1 linha(s) retornada(s).

Mensagem do MySQL ao tentar o INSERT:

```
ERROR 1644 (45000): Turma lotada! Nao ha vagas disponiveis.
```

6 CONSIDERAÇÕES FINAIS

Fazer esse projeto ajudou a gente a entender na prática por que a modelagem tem uma ordem. Quando tentamos começar direto pelas tabelas, várias coisas ficaram erradas, como o plano dentro do aluno e os exercícios como um atributo da ficha, e só percebemos isso quando voltamos para o MER e pensamos nas cardinalidades com calma.

As partes mais difíceis foram entender quando um relacionamento vira tabela associativa, lembrar de colocar no GROUP BY as colunas que aparecem no SELECT junto com funções de agregação, e escrever os triggers e procedures, principalmente por causa do DELIMITER. Também demoramos um pouco para entender a diferença entre INNER JOIN e LEFT JOIN, que só ficou clara quando fizemos a consulta dos alunos sem ficha de treino.

No final, o banco ficou com 11 tabelas, 195 registros de teste, 31 consultas, 2 triggers e 3 stored procedures, cobrindo todos os comandos pedidos no trabalho. Como melhorias para o futuro, daria para incluir a avaliação física dos alunos, guardar o histórico de preços dos planos e criar usuários com permissões diferentes usando comandos DCL (GRANT e REVOKE), por exemplo um usuário só de consulta para a recepção.

REFERÊNCIAS

CÂNDIDO, Carlos Henrique. **brModelo**: ferramenta de modelagem de dados. Versão 3.31. [S. l.], 2020. Disponível em: https://github.com/chcandido/brModelo. Acesso em: 8 nov. 2026.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson Education do Brasil, 2018.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

ORACLE. **MySQL 8.0 Reference Manual**. [S. l.]: Oracle, 2026a. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 8 nov. 2026.

ORACLE. **MySQL Workbench Manual**. [S. l.]: Oracle, 2026b. Disponível em: https://dev.mysql.com/doc/workbench/en/. Acesso em: 8 nov. 2026.

PROGRAMADORES DEPRÊ. **Comandos SQL: DDL, DML, DCL, TCL, DQL e cláusulas**. [S. l.], 2026. Disponível em: https://programadoresdepre.com.br/comandos-sql-ddl-dml-dcl-tcl-dql-e-clausulas/. Acesso em: 8 nov. 2026.

SOUZA, Josyane Lannes Florenzano de. **Projeto final de modelagem de dados**. 2026. Material de aula (apresentação de slides) da disciplina Modelagem de Banco de Dados.

W3SCHOOLS. **SQL Tutorial**. [S. l.], 2026. Disponível em: https://www.w3schools.com/sql/. Acesso em: 8 nov. 2026.

APÊNDICE A – SCRIPT DE CRIAÇÃO DO BANCO

Script completo do arquivo `01_criacao_tabelas.sql`, que cria o banco e as tabelas:

```
-- ======================================================================
-- PROJETO FINAL - MODELAGEM DE BANCO DE DADOS
-- Tema: Sistema de Gerenciamento da Academia BoaForma
-- Arquivo 01: Criacao do banco e das tabelas (DDL)
-- ======================================================================
-- DDL = Data Definition Language (linguagem de definicao de dados).
-- Sao os comandos que criam/alteram/apagam a ESTRUTURA do banco:
-- CREATE, ALTER, DROP.
-- ======================================================================

-- Se o banco ja existir, apaga pra comecar do zero (bom pra testar varias vezes)
DROP DATABASE IF EXISTS academia_boaforma;

-- Cria o banco de dados da academia
CREATE DATABASE academia_boaforma
    DEFAULT CHARACTER SET utf8mb4
    DEFAULT COLLATE utf8mb4_general_ci;

-- Fala pro MySQL que daqui pra frente vamos usar esse banco
USE academia_boaforma;

-- ----------------------------------------------------------------------
-- Tabela PLANO
-- Guarda os planos que a academia vende (mensal, trimestral...)
-- ----------------------------------------------------------------------
CREATE TABLE plano (
    id_plano       INT           NOT NULL AUTO_INCREMENT, -- numero gerado sozinho
    nome           VARCHAR(50)   NOT NULL,
    descricao      VARCHAR(200),
    valor_mensal   DECIMAL(8,2)  NOT NULL,                -- 8 digitos, 2 depois da virgula
    duracao_meses  INT           NOT NULL,
    PRIMARY KEY (id_plano)
);

-- ----------------------------------------------------------------------
-- Tabela ALUNO
-- Cada aluno assina UM plano (por isso tem a chave estrangeira id_plano)
-- O endereco ficou em colunas simples: rua, numero, bairro e cidade
-- ----------------------------------------------------------------------
CREATE TABLE aluno (
    id_aluno         INT           NOT NULL AUTO_INCREMENT,
    nome             VARCHAR(100)  NOT NULL,
    cpf              CHAR(14)      NOT NULL,             -- formato 000.000.000-00
    data_nascimento  DATE          NOT NULL,
    sexo             CHAR(1),                            -- 'M' ou 'F'
    telefone         VARCHAR(15),
    email            VARCHAR(100),
    rua              VARCHAR(100),
    numero           VARCHAR(10),
    bairro           VARCHAR(50),
    cidade           VARCHAR(50),
    data_matricula   DATE          NOT NULL,
    situacao         VARCHAR(10)   NOT NULL DEFAULT 'ATIVO', -- ATIVO ou INATIVO
    id_plano         INT           NOT NULL,
    PRIMARY KEY (id_aluno),
    UNIQUE (cpf),                                        -- nao pode ter dois alunos com o
mesmo CPF
    FOREIGN KEY (id_plano) REFERENCES plano (id_plano)
);

-- ----------------------------------------------------------------------
-- Tabela INSTRUTOR
-- Os professores da academia. O CREF e o registro do profissional de
-- Educacao Fisica (tipo o CRMV do veterinario no exemplo do zoologico)
```

```
-- ---------------------------------------------------------------------
CREATE TABLE instrutor (
    id_instrutor   INT           NOT NULL AUTO_INCREMENT,
    nome           VARCHAR(100)  NOT NULL,
    cpf            CHAR(14)      NOT NULL,
    cref           VARCHAR(20)   NOT NULL,
    telefone       VARCHAR(15),
    email          VARCHAR(100),
    salario        DECIMAL(8,2)  NOT NULL,
    data_admissao  DATE          NOT NULL,
    PRIMARY KEY (id_instrutor),
    UNIQUE (cpf),
    UNIQUE (cref)
);

-- ---------------------------------------------------------------------
-- Tabela MODALIDADE
-- Tipos de aula: musculacao, spinning, pilates, etc.
-- ---------------------------------------------------------------------
CREATE TABLE modalidade (
    id_modalidade  INT           NOT NULL AUTO_INCREMENT,
    nome           VARCHAR(50)   NOT NULL,
    descricao      VARCHAR(200),
    PRIMARY KEY (id_modalidade),
    UNIQUE (nome)
);

-- ---------------------------------------------------------------------
-- Tabela TURMA
-- Uma turma e uma aula de uma modalidade, num dia e horario,
-- dada por um instrutor. Tem um limite de vagas.
-- ---------------------------------------------------------------------
CREATE TABLE turma (
    id_turma       INT          NOT NULL AUTO_INCREMENT,
    dia_semana     VARCHAR(15)  NOT NULL,
    horario        TIME         NOT NULL,
    vagas          INT          NOT NULL,
    id_modalidade  INT          NOT NULL,
    id_instrutor   INT          NOT NULL,
    PRIMARY KEY (id_turma),
    FOREIGN KEY (id_modalidade) REFERENCES modalidade (id_modalidade),
    FOREIGN KEY (id_instrutor)  REFERENCES instrutor (id_instrutor)
);

-- ---------------------------------------------------------------------
-- Tabela ALUNO_TURMA (tabela associativa)
-- Veio do relacionamento N:N "participa" entre ALUNO e TURMA.
-- A chave primaria e composta pelas duas chaves estrangeiras.
-- ---------------------------------------------------------------------
CREATE TABLE aluno_turma (
    id_aluno        INT   NOT NULL,
    id_turma        INT   NOT NULL,
    data_inscricao  DATE  NOT NULL,
    PRIMARY KEY (id_aluno, id_turma),
    FOREIGN KEY (id_aluno) REFERENCES aluno (id_aluno),
    FOREIGN KEY (id_turma) REFERENCES turma (id_turma)
);

-- ---------------------------------------------------------------------
-- Tabela PAGAMENTO
-- Cada pagamento e de um aluno, referente a um mes (ex: '2026-10')
-- ---------------------------------------------------------------------
CREATE TABLE pagamento (
    id_pagamento     INT           NOT NULL AUTO_INCREMENT,
    data_pagamento   DATE          NOT NULL,
    mes_referencia   CHAR(7)       NOT NULL,             -- formato AAAA-MM
    valor            DECIMAL(8,2)  NOT NULL,
    forma_pagamento  VARCHAR(20)   NOT NULL,             -- PIX, DINHEIRO, CARTAO...
    id_aluno         INT           NOT NULL,
    PRIMARY KEY (id_pagamento),
    FOREIGN KEY (id_aluno) REFERENCES aluno (id_aluno)
);

-- ---------------------------------------------------------------------
```

```
-- Tabela FICHA_TREINO
-- A ficha que o instrutor monta pro aluno
-- ---------------------------------------------------------------------
CREATE TABLE ficha_treino (
    id_ficha      INT           NOT NULL AUTO_INCREMENT,
    data_criacao  DATE          NOT NULL,
    objetivo      VARCHAR(100)  NOT NULL,
    id_aluno      INT           NOT NULL,
    id_instrutor  INT           NOT NULL,
    PRIMARY KEY (id_ficha),
    FOREIGN KEY (id_aluno)     REFERENCES aluno (id_aluno),
    FOREIGN KEY (id_instrutor) REFERENCES instrutor (id_instrutor)
);


-- ---------------------------------------------------------------------
-- Tabela EXERCICIO
-- Lista de exercicios que podem entrar numa ficha
-- ---------------------------------------------------------------------
CREATE TABLE exercicio (
    id_exercicio    INT          NOT NULL AUTO_INCREMENT,
    nome            VARCHAR(60)  NOT NULL,
    grupo_muscular  VARCHAR(30)  NOT NULL,
    PRIMARY KEY (id_exercicio)
);


-- ---------------------------------------------------------------------
-- Tabela FICHA_EXERCICIO (tabela associativa)
-- Veio do relacionamento N:N "contem" entre FICHA_TREINO e EXERCICIO.
-- Series, repeticoes e carga sao atributos do relacionamento, porque
-- dependem da ficha E do exercicio ao mesmo tempo.
-- ---------------------------------------------------------------------
CREATE TABLE ficha_exercicio (
    id_ficha      INT           NOT NULL,
    id_exercicio  INT           NOT NULL,
    series        INT           NOT NULL,
    repeticoes    INT           NOT NULL,
    carga_kg      DECIMAL(5,1),
    PRIMARY KEY (id_ficha, id_exercicio),
    FOREIGN KEY (id_ficha)     REFERENCES ficha_treino (id_ficha),
    FOREIGN KEY (id_exercicio) REFERENCES exercicio (id_exercicio)
);


-- ---------------------------------------------------------------------
-- Tabela FREQUENCIA
-- Registra cada vez que o aluno passa na catraca
-- ---------------------------------------------------------------------
CREATE TABLE frequencia (
    id_frequencia  INT   NOT NULL AUTO_INCREMENT,
    data_entrada   DATE  NOT NULL,
    hora_entrada   TIME  NOT NULL,
    id_aluno       INT   NOT NULL,
    PRIMARY KEY (id_frequencia),
    FOREIGN KEY (id_aluno) REFERENCES aluno (id_aluno)
);

-- Mostra as tabelas criadas so pra conferir
SHOW TABLES;
```

APÊNDICE B – COMO RODAR O PROJETO NA APRESENTAÇÃO

Passo a passo que usamos para mostrar o banco funcionando em sala:

1. Conferir se o MySQL Server está ligado. No Mac com Homebrew: `brew services start mysql`.
2. Abrir o MySQL Workbench e clicar na conexão **Local instance 3306** (usuário root).
3. Em **File > Open SQL Script**, abrir o `01_criacao_tabelas.sql` e executar tudo pelo botão do raio (ou Ctrl+Shift+Enter). O banco e as 11 tabelas são criados do zero.
4. Abrir e executar o `02_insercao_dados.sql`. No final ele mostra quantos registros ficaram em cada tabela.
5. Abrir o `03_consultas.sql`, clicar em cada consulta e apertar Ctrl+Enter (Cmd+Enter no Mac) para rodar só aquela consulta e mostrar o resultado.
6. Para os triggers e procedures, selecionar no `03_consultas.sql` o trecho "4.3 CONSULTAS AVANCADAS" (do primeiro DROP TRIGGER até o último DELIMITER ;) e executar só a seleção. Depois rodar os testes das consultas 27 a 31. A consulta 31 dá erro de propósito.
7. Para mostrar os diagramas: o MER abre no brModelo (Arquivo > Abrir > `diagramas/MER_Academia_BoaForma.brM3`) e o DER abre no Workbench (File > Open Model > `diagramas/DER_Academia_BoaForma.mwb`).
8. Para começar tudo de novo, é só rodar os arquivos 01 e 02 outra vez.